

1. A genealogia das linguagens não é uma escada de progresso. Explique essa afirmação e apresente dois fatores históricos que fazem uma linguagem influenciar outra sem necessariamente substituí-la.

Essa afirmação significa que a evolução das linguagens de programação não segue uma linha reta em que cada linguagem nova é simplesmente "melhor" e substitui a anterior, como degraus de uma escada. Na prática, linguagens antigas continuam vivas, coexistem com as novas e continuam influenciando o design de linguagens futuras mesmo décadas depois, existe uma árvore de influências, não uma hierarquia de superação.

Dois fatores históricos que explicam isso:

- **Investimento em código legado e treinamento de programadores:** empresas e instituições que já tinham grandes bases de código e equipes treinadas em uma linguagem não abandonam essa linguagem só porque uma nova surge. A nova linguagem influencia conceitos, mas a antiga permanece em uso por razões econômicas e práticas.
- **Nichos de aplicação diferentes:** linguagens surgem para resolver problemas específicos. Uma linguagem nova pode incorporar ideias de outra (por exemplo, Lisp influenciando linguagens funcionais modernas) sem que a linguagem original deixe de ser usada originalmente.

2. Plankalkül não foi implementada em sua época. Ainda assim, por que ela é relevante para a história das linguagens? Cite três recursos antecipados por seu projeto e explique o valor de um deles.

Plankalkül, nunca foi implementada na época porque não tinha computador disponível para executá-la e o trabalho ficou obscuro até muito depois. Mesmo assim, é relevante porque antecipou conceitos que só apareceriam formalmente em linguagens muito posteriores.

Três recursos antecipados:

- Tipos de dados estruturados (arrays e registros).
- Atribuição e estruturas de repetição/controle de fluxo.
- Um conceito rudimentar de invariantes/asserções para verificar estados do programa.

Valor de um deles: os tipos de dados estruturados são particularmente importantes porque mostram que, mesmo antigamente, havia a percepção de que programar não era só manipular números soltos, mas organizar dados em estruturas, uma ideia que anos depois fundamentaria linguagens como Pascal e C.

3. Compare Short Code, Speedcoding e os sistemas A-0/A-1/A-2 quanto ao problema enfrentado e à estratégia adotada. Por que chamá-los simplesmente de compiladores modernos seria impreciso?

- **Short Code:** era um pseudocódigo interpretado, o programador escrevia expressões matemáticas que eram interpretadas linha a linha pela máquina. O problema enfrentado era a dificuldade de programar diretamente em código de máquina para cálculos matemáticos.
- **Speedcoding:** também era um interpretador, criado para a IBM 701, oferecendo operações de ponto flutuante e funções matemáticas built-in, priorizando facilidade de uso mesmo com grande perda de desempenho (interpretação lenta).
- **Sistemas A-0, A-1, A-2:** representam um passo além, não eram interpretadores, mas sistemas que geravam/ligavam sub-rotinas já compiladas em código de máquina a partir de um código mais simples, uma estratégia mais próxima de um compilador real, embora ainda limitada.

Chamá-los simplesmente de "compiladores modernos" seria impreciso porque Short Code e Speedcoding eram interpretadores, e mesmo os sistemas A-0/A-1/A-2, apesar de mais próximos da ideia de compilação, ainda eram rudimentares, funcionando mais como ligadores de sub-rotinas do que como compiladores completos que entendemos hoje (que só surgiria definitivamente com Fortran).

4. Explique por que o projeto Fortran precisou convencer programadores de que código traduzido podia competir com código de máquina escrito à mão. Relacione desempenho, custo de programação e adoção.

Nos anos 1950, os programadores escreviam código de máquina ou assembly diretamente, otimizando manualmente cada instrução, pois computadores eram caros e lentos, e desperdiçar ciclos de processamento com um "código traduzido" ineficiente parecia inaceitável. Havia forte ceticismo de que um compilador pudesse gerar código tão eficiente quanto um programador experiente.

A equipe do Fortran precisou provar que:

- **Desempenho:** o código gerado pelo compilador Fortran chegava perto (ou igualava) à eficiência do código de máquina escrito à mão, através de otimizações agressivas no compilador.
- **Custo de programação:** mesmo que o código gerado fosse um pouco menos eficiente, o tempo economizado em escrever, depurar e manter programas em uma linguagem de alto nível compensava largamente, pois o custo humano em horas de programação começava a pesar mais que o custo de máquina.
- **Adoção:** só quando ficou demonstrado que a relação custo-benefício era favorável (código rápido o suficiente + muito menos tempo de desenvolvimento) é que o Fortran ganhou aceitação ampla, abrindo caminho para a era das linguagens de alto nível.

5. Lisp surgiu em um contexto diferente de Fortran. Compare os domínios, a representação de dados e o estilo de computação favorecido pelas duas linguagens.

- **Domínio:** Fortran foi criada para computação científica/numérica (cálculos de engenharia, física). Lisp foi criada para pesquisa em inteligência artificial, processamento simbólico e manipulação de listas.
- **Representação de dados:** Fortran usa principalmente arrays e variáveis numéricas de tipos fixos. Lisp usa listas como estrutura de dados universal, permitindo representar tanto dados quanto código na mesma forma.
- **Estilo de computação:** Fortran favorece um estilo imperativo, com atribuições, laços e fluxo sequencial de controle. Lisp favorece um estilo funcional, baseado em recursão e aplicação de funções, com menor ênfase em estado mutável.

10. Defina ortogonalidade no projeto de linguagens e use ALGOL 68 para discutir a diferença entre regularidade e simplicidade. Uma linguagem muito ortogonal é automaticamente fácil de usar?

Ortogonalidade no projeto de linguagens significa que um pequeno conjunto de construções primitivas pode ser combinado livremente, de forma consistente e sem restrições arbitrárias, para formar estruturas mais complexas. Cada elemento da linguagem é independente e pode se combinar com qualquer outro de maneira previsível.

ALGOL 68 é o exemplo clássico de linguagem extremamente ortogonal: praticamente qualquer combinação de tipos, expressões e estruturas era permitida, tornando a linguagem muito regular (poucas exceções às regras). Isso ilustra a diferença entre regularidade e simplicidade: regularidade é a ausência de casos especiais (a linguagem é previsível e consistente). Simplicidade é a facilidade de aprender e usar a linguagem. ALGOL 68 era altamente regular, mas isso gerou uma explosão combinatória de possibilidades que tornou a linguagem difícil de entender, ler e implementar.

Portanto, não: uma linguagem muito ortogonal não é automaticamente fácil de usar. Regularidade excessiva pode gerar complexidade cognitiva, porque o programador precisa lidar com um número muito grande de combinações válidas, e erros sutis tornam-se mais difíceis de detectar.

11. Construa uma cadeia de influência que passe por ALGOL, Pascal e C. Depois contraste essa linhagem imperativa com a proposta declarativa de Prolog.

- **ALGOL** (60/68) introduziu estruturas de bloco, tipagem mais rigorosa e a ideia de sintaxe formalmente definida (BNF), influenciando praticamente todas as linguagens imperativas seguintes.
- **Pascal**, criada por Niklaus Wirth, herdou de ALGOL a estrutura em blocos e a ênfase em programação estruturada, mas simplificou a linguagem, tornando-a mais voltada ao ensino e à clareza.
- **C**, criada por Dennis Ritchie, também herdou conceitos estruturais de ALGOL (via B/BCPL), mas priorizou eficiência, controle de baixo nível sobre memória e proximidade com hardware — características que tornaram C a base de sistemas operacionais como Unix.

Essa linhagem — ALGOL, Pascal, C — é fundamentalmente **imperativa**: o programador especifica passo a passo *como* o problema deve ser resolvido, com atribuições, laços e sequência de comandos que alteram o estado da máquina.

Prolog, em contraste, é **declarativa** (lógica): o programador especifica *o que* é verdadeiro (fatos e regras) e deixa que o mecanismo de inferência da linguagem (resolução e unificação) determine *como* chegar à resposta, através de consultas. Não há um fluxo de controle explícito imposto pelo programador da mesma forma que nas linguagens imperativas.

12. Modele em linguagem natural uma pequena base Prolog com dois fatos, uma regra e uma consulta. Explique por que isso representa programação lógica, não apenas armazenamento de dados.

Fatos:

- "Maria é mãe de João."
- "João é pai de Pedro."

Regra:

- "X é avó de Y se X é mãe de Z e Z é pai de Y."

Consulta:

- "Quem é avó de Pedro?"

O sistema, ao processar essa consulta, tentaria unificar a regra com os fatos disponíveis: encontraria que João é pai de Pedro, depois buscaria quem é mãe de João (Maria), concluindo que Maria é avó de Pedro.

Isso representa programação lógica, e não apenas armazenamento de dados, porque a resposta "Maria é avó de Pedro" **não foi armazenada explicitamente** em lugar nenhum da base — ela foi **inferida** pelo motor de inferência a partir da combinação de fatos e regras. Um banco de dados tradicional apenas recupera o que foi gravado; Prolog deriva novos conhecimentos por dedução lógica.

13. Ada resultou de requisitos e projeto em grande escala. Analise como confiabilidade, tipos, pacotes e concorrência se relacionam ao domínio de sistemas críticos.

Ada foi encomendada pelo Departamento de Defesa dos EUA para unificar centenas de linguagens usadas em sistemas embarcados militares, priorizando confiabilidade em ambientes onde falhas podem ser catastróficas (armamentos, aviônica, controle de mísseis).

- **Confiabilidade:** Ada foi projetada com verificação rigorosa em tempo de compilação e de execução, reduzindo erros que poderiam passar despercebidos em sistemas críticos.

- **Tipos:** um sistema de tipos fortemente estático, com subtipos e faixas de valores explícitas, permite detectar erros de uso incorreto de dados antes mesmo da execução — essencial quando um erro de tipo pode significar falha de um sistema físico.
- **Pacotes:** o mecanismo de pacotes (packages) permite encapsulamento e modularização, facilitando o desenvolvimento e manutenção de sistemas grandes e complexos por múltiplas equipes, com interfaces bem definidas.
- **Concorrência:** o suporte nativo a tarefas concorrentes (tasks) foi incluído porque sistemas embarcados militares frequentemente precisam lidar com múltiplos processos em tempo real simultaneamente (sensores, atuadores, comunicação), e a linguagem precisava oferecer essa capacidade de forma secura e padronizada, sem depender de bibliotecas externas.

Juntos, esses elementos refletem diretamente as exigências do domínio: sistemas onde uma falha de software pode ter consequências físicas graves, em ambientes com múltiplos processos concorrentes e forte necessidade de manutenção em longo prazo por grandes equipes.

14. Compare o papel dos objetos em Smalltalk, C++ e Java. Inclua na resposta o compromisso de C++ com C e a estratégia de portabilidade de Java.

- **Smalltalk:** linguagem puramente orientada a objetos desde sua concepção — tudo é objeto, incluindo tipos primitivos, e toda computação ocorre por troca de mensagens entre objetos. Foi projetada do zero para esse paradigma, sem compromissos com linguagens anteriores.
- **C++:** adicionou orientação a objetos a C, mas manteve compatibilidade retroativa com C (compromisso com C), permitindo que programadores escrevessem tanto código puramente procedural quanto orientado a objetos. Isso trouxe flexibilidade e facilidade de adoção por programadores C já existentes, mas também gerou uma linguagem híbrida, mais complexa, com múltiplos paradigmas coexistindo.
- **Java:** foi projetada para ser orientada a objetos de forma mais disciplinada que C++ (sem herança múltipla direta, sem ponteiros explícitos), e adotou uma estratégia de portabilidade baseada na JVM (Java Virtual Machine) — o código é compilado para bytecode intermediário, que roda em qualquer plataforma com uma JVM instalada, seguindo o lema "escreva uma vez, rode em qualquer lugar" ("write once, run anywhere").

Assim, Smalltalk representa a OO "pura", C++ representa OO como extensão pragmática de uma linguagem procedural existente, e Java representa OO combinada com portabilidade multiplataforma como estratégia central de design.

15. A primeira aplicação de Java não foi a Web, mas a Web impulsionou sua adoção. Explique como mudanças de contexto podem reposicionar uma linguagem.

A primeira aplicação-alvo de Java (então chamada Oak) foi eletrônicos de consumo embarcados (como controladores para dispositivos domésticos), um mercado que não decolou como esperado na época. No entanto, quando a World Wide Web começou a crescer explosivamente em meados dos anos 1990, características que já existiam em Java — portabilidade via bytecode/JVM, segurança e a possibilidade de rodar pequenos

programas (applets) dentro de navegadores — encaixaram-se perfeitamente nas necessidades emergentes da Web, como conteúdo interativo em páginas.

Isso ilustra como uma linguagem pode ser reposicionada por mudanças de contexto: as propriedades técnicas de Java não mudaram, mas o cenário tecnológico e as necessidades do mercado sim, e características projetadas para um domínio (dispositivos embarcados) acabaram se tornando decisivas para o sucesso em outro domínio completamente diferente (aplicações web), que a impulsionou à popularidade massiva.